

Java Backend / SDE — 90 Day Complete Learning Roadmap

Goal: move from assisted task completion to independent, job-ready backend engineering.

Definition of Done: A topic is complete only when you can explain it, implement it, use it in a project, debug a failure, and answer interview-level questions.

How to Use This Roadmap

- Main study: 2–3 hours/day on the phase topic.
- DSA: 45–60 minutes/day in parallel.
- Project: 30–60 minutes/day connecting the topic to a real backend system.
- Daily New Knowledge: 15–20 minutes; this is intentionally outside the main syllabus.
- Every 7th day: revision + self-test; do not simply move forward with gaps.
- Use ChatGPT as a mentor/debugging partner: attempt first, then ask for hints or review.

Phase 1 — Core Java Foundations (Days 1–15)

1. Java execution model: JDK/JRE/JVM, compilation, bytecode, classpath, packages
2. OOP: classes, objects, constructors, encapsulation, inheritance, polymorphism, abstraction
3. Interfaces vs abstract classes, composition vs inheritance, SOLID basics
4. String, String pool, immutability, StringBuilder/StringBuffer
5. equals(), hashCode(), toString(), object identity
6. Collections overview, Iterable/Collection/List/Set/Map/Queue
7. ArrayList, LinkedList, Vector, CopyOnWriteArrayList basics
8. HashSet, LinkedHashSet, TreeSet and ordering
9. HashMap internals, hashing, collisions, resize, load factor, tree bins
10. LinkedHashMap, TreeMap, Comparator and Comparable
11. Generics, wildcards, bounded types, type erasure
12. Exceptions: checked/unchecked, custom exceptions, try-with-resources
13. Java 8+: lambda, functional interfaces, method references, Optional
14. Stream API: map, filter, flatMap, reduce, collect, grouping, partitioning
15. Mini assessment: implement utilities using collections, streams and clean OOP

Phase 2 — Advanced Java & Concurrency (Days 16–25)

1. Thread lifecycle, Runnable, Callable, Future
2. ExecutorService, thread pools, scheduling
3. synchronized, intrinsic locks, monitor concept
4. volatile, visibility, happens-before basics
5. AtomicInteger/AtomicReference and CAS concept
6. ReentrantLock, ReadWriteLock, Semaphore
7. Race conditions, deadlocks, starvation and livelock
8. ConcurrentHashMap and concurrent collections
9. CompletableFuture and asynchronous composition
10. JVM memory: stack, heap, metaspace, references and common memory issues

Phase 3 — Spring & Spring Boot (Days 26–40)

1. Spring IoC, DI, ApplicationContext and Bean lifecycle
2. Bean scopes, component scanning, @Bean vs @Component
3. Spring Boot starters, auto-configuration and configuration properties
4. application.yml/properties, profiles and environment configuration
5. REST API design: resources, HTTP methods, status codes and error contracts
6. Controller-Service-Repository architecture and DTO mapping
7. Validation with Bean Validation and global exception handling
8. Logging, correlation IDs and structured logs
9. Spring Data JPA repositories and query methods
10. Hibernate persistence context, dirty checking and entity lifecycle
11. Relationships: OneToOne, OneToMany, ManyToOne, ManyToMany
12. Lazy vs eager loading, N+1 problem and fetch strategies

13. Transactions: @Transactional, propagation and rollback basics
14. Pagination, sorting, specifications and dynamic queries
15. Spring Security: authentication, authorization, JWT, RBAC and OAuth2 basics

Phase 4 — Database & Persistence (Days 41–50)

1. Relational modeling, primary/foreign keys and constraints
2. SELECT, INSERT, UPDATE, DELETE and joins
3. Subqueries, CTEs and window functions
4. Indexes: B-tree, selectivity, composite indexes and covering concepts
5. EXPLAIN/EXPLAIN ANALYZE and query-plan reasoning
6. Transactions and ACID
7. Isolation levels, locking and deadlocks
8. Normalization vs denormalization
9. Connection pooling and database performance
10. PostgreSQL/TimescaleDB concepts, migrations with Liquibase/Flyway

Phase 5 — Microservices, Kafka & Redis (Days 51–62)

1. Monolith vs microservices and service boundaries
2. API Gateway, service discovery and Eureka
3. REST vs synchronous messaging vs asynchronous events
4. Inter-service communication, timeouts, retries and circuit breakers
5. Idempotency and distributed failure handling
6. Kafka architecture: broker, topic, partition, replica
7. Producer acknowledgements, serialization and partitioning
8. Consumer groups, offsets, rebalancing and ordering
9. Retries, DLQ, delivery semantics and duplicate messages
10. Kafka lag, scaling consumers and operational troubleshooting
11. Redis data structures, caching and TTL
12. Cache-aside, invalidation, cache stampede basics and Redis failure handling

Phase 6 — DevOps & Production Engineering (Days 63–68)

1. Git branching, merge/rebase, cherry-pick, stash and conflict resolution
2. Maven lifecycle, dependency management, profiles and build artifacts
3. Testing: JUnit, Mockito, integration-test strategy
4. Dockerfile, image layers, containers, volumes and networks
5. Docker Compose, environment files, health checks and service dependencies
6. Jenkins CI/CD: checkout → build → test → scan → image → registry → deployment

Phase 7 — System Design & Architecture (Days 69–75)

1. Functional/non-functional requirements and capacity estimation
2. API design and database schema for scalable systems
3. Load balancing, horizontal scaling and stateless services
4. Caching strategies and cache consistency

5. Message queues, event-driven architecture and backpressure
6. Rate limiting, idempotency and resilience patterns
7. Design exercises: URL shortener, notification system, order system, SIEM/log pipeline

Phase 8 — Real Backend Project (Days 76–84)

1. Project architecture and requirements
2. Authentication, JWT and RBAC
3. User/service CRUD with PostgreSQL
4. Validation, exception handling and API documentation
5. Redis caching and performance measurement
6. Kafka event pipeline with retry/DLQ
7. Search/document processing or telemetry-style asynchronous workflow
8. Dockerize all services and create Docker Compose environment
9. Add tests, observability, health checks and production-style configuration

Phase 9 — Independent Developer & Interview Mode (Days 85–90)

1. Debugging drills: logs, stack traces, database and network diagnosis
2. Production incident simulation: latency, CPU, memory, Kafka lag, DB bottleneck
3. Project deep dive and architecture explanation
4. Java/Spring/backend rapid-fire interview
5. DSA timed coding round
6. Full mock interview + final gap analysis

Parallel DSA Track — Days 1–90

- DSA track runs in parallel every day: 45–60 minutes.
- Days 1–10: arrays, strings, hashing, frequency maps, basic complexity.
- Days 11–18: two pointers, sliding window, prefix sum, binary search.
- Days 19–26: stack, queue, deque, linked list.
- Days 27–36: recursion, trees, BST, traversals.
- Days 37–45: heap, priority queue, intervals, greedy basics.
- Days 46–55: graphs, BFS, DFS, topological sort, shortest-path basics.
- Days 56–65: dynamic programming fundamentals and common patterns.
- Days 66–75: mixed medium problems and pattern recognition.
- Days 76–84: timed interview sets.
- Days 85–90: revision, weak-pattern drills and mock coding rounds.
- Target: ~150 quality problems with notes on pattern, approach, complexity and mistakes.

Daily New Knowledge Track — 90 Topics

These are short curiosity/engineering topics designed to expand your general software-engineering knowledge without derailing the main syllabus.

Day	New knowledge topic
1	1: Big-O and why it matters in backend engineering
2	2: How a Java source file becomes bytecode and runs
3	3: SOLID in real backend code
4	4: Immutability and why it improves reliability
5	5: Hash collisions and real-world hashing
6	6: Why LinkedList is often a poor default
7	7: Tree data structures and ordered lookup
8	8: HashMap resize and performance impact
9	9: Type erasure in Java generics
10	10: Why checked exceptions are controversial
11	11: Functional programming ideas in Java
12	12: Stream laziness and short-circuiting
13	13: Why parallel streams can hurt performance
14	14: Resource management and try-with-resources
15	15: How to review your own Java code
16	16: Threads vs processes
17	17: Thread pool sizing intuition
18	18: Memory visibility and the Java Memory Model
19	19: CAS and lock-free programming
20	20: Deadlocks in real systems
21	21: Backpressure in concurrent systems
22	22: CompletableFuture execution model
23	23: Why shared mutable state is dangerous
24	24: JVM garbage collection at a high level
25	25: How to investigate a Java memory leak
26	26: Dependency Injection as a design pattern
27	27: Spring Bean lifecycle
28	28: Spring Boot auto-configuration
29	29: Configuration management and the 12-factor idea
30	30: REST resource design
31	31: DTOs vs entities
32	32: API error design and consistent contracts
33	33: Correlation IDs and distributed debugging
34	34: JPA persistence context
35	35: Dirty checking
36	36: N+1 queries
37	37: Transaction boundaries
38	38: JWT anatomy
39	39: Authentication vs authorization
40	40: RBAC and least privilege
41	41: Database normalization trade-offs
42	42: Composite index column order

Day	New knowledge topic
43	43: Reading an EXPLAIN plan
44	44: ACID in practical systems
45	45: Isolation levels and anomalies
46	46: Deadlocks and lock ordering
47	47: Connection pool exhaustion
48	48: Why ORMs can create hidden queries
49	49: Time-series database concepts
50	50: Database migrations and safe schema changes
51	51: Why microservice boundaries matter
52	52: Service discovery
53	53: API Gateway responsibilities
54	54: Timeout/retry/circuit-breaker interaction
55	55: Idempotency keys
56	56: Kafka partitions and ordering
57	57: Consumer groups and scaling
58	58: Kafka offset management
59	59: At-least-once delivery and duplicates
60	60: Dead-letter queues
61	61: Consumer lag diagnosis
62	62: Cache invalidation
63	63: Git rebase vs merge
64	64: Maven dependency conflicts
65	65: Unit vs integration tests
66	66: Docker image layers
67	67: Container networking
68	68: CI/CD as a feedback loop
69	69: CAP theorem intuition
70	70: Horizontal vs vertical scaling
71	71: Load balancing
72	72: Cache consistency
73	73: Rate limiting algorithms
74	74: Event-driven architecture
75	75: Observability: logs, metrics, traces
76	76: Production configuration and secrets
77	77: Health checks vs readiness checks
78	78: API versioning
79	79: Distributed tracing
80	80: Feature flags
81	81: Graceful shutdown
82	82: Resilience testing / failure injection
83	83: SLO, SLA and error budgets
84	84: Post-incident review and root-cause analysis
85	85: How senior engineers debug unfamiliar code
86	86: Reading stack traces efficiently
87	87: Production latency decomposition

Day	New knowledge topic
88	88: Technical trade-offs and ADRs
89	89: How to communicate an incident
90	90: Building a personal engineering knowledge base

Daily Execution Template

1. Learn: understand the concept from reliable material and examples.
2. Recall: close the material and explain it in your own words.
3. Build: write a small implementation without copying.
4. Apply: connect it to a real backend use case.
5. Debug: intentionally inspect an error/performance/failure scenario.
6. DSA: solve 1–3 problems based on the current pattern.
7. New knowledge: spend 15–20 minutes on the day's extra topic.
8. Revision: write 5–10 bullets from memory.
9. Interview: answer 5–10 questions aloud.
10. Mark the topic Done only if you can work without step-by-step assistance.

Final 90-Day Outcome

- Independently create and structure a Spring Boot backend service.
- Design REST APIs, validation, error handling, security and persistence.
- Work confidently with PostgreSQL/JPA and diagnose common performance problems.
- Build and reason about Kafka-based asynchronous workflows.
- Use Redis for practical caching scenarios.
- Containerize services and understand a production CI/CD pipeline.
- Explain and design practical microservice/backend architectures.
- Solve common DSA patterns at interview level.
- Debug unfamiliar Java backend code systematically.
- Explain your real project architecture and technical decisions without relying on memorized answers.